Quantitative Finance Foundations

A Practitioner's Guide to Stochastic Models, Options Pricing, and Risk Analytics

Covering: Stock Returns • Stochastic Processes • GBM • Jump-Diffusion • Heston
Black-Scholes • Monte Carlo Methods • Greeks • Variance Reduction • Risk Metrics

Companion course for the Monte Carlo Options Pricing System

Table of Contents

Chapter 1 — Foundations: Stock Prices and Returns

Chapter 2 — Probability and Random Variables

Chapter 3 — Stochastic Processes and Brownian Motion

Chapter 4 — Geometric Brownian Motion (GBM)

Chapter 5 — Merton Jump-Diffusion Model

Chapter 6 — Heston Stochastic Volatility Model

Chapter 7 — Options: Contracts, Payoffs, and Strategies

Chapter 8 — Black-Scholes: The Closed-Form Benchmark

Chapter 9 — Monte Carlo Simulation for Pricing

Chapter 10 — Variance Reduction Techniques

Chapter 11 — The Greeks: Sensitivity Analysis

Chapter 12 — Risk Metrics: VaR, CVaR, and Beyond

Chapter 13 — Portfolio Simulation and Correlation

Chapter 14 — Model Calibration from Market Data

Chapter 15 — Stress Testing and Scenario Analysis

Appendix A — Mathematical Reference

Chapter 1: Foundations — Stock Prices and Returns

1.1 What Is a Stock Price?

A stock price represents the current market value of one share of a company. Prices are determined by supply and demand in exchanges. For quantitative modeling, we work with **adjusted close prices** which account for stock splits and dividend distributions, giving us a continuous, comparable time series.

When you see raw OHLCV data (Open, High, Low, Close, Volume), the adjusted close is the column that matters for modeling. A 2-for-1 split halves the raw price but adjusted close retroactively corrects all historical prices so the series remains smooth.

1.2 Simple Returns vs. Log Returns

There are two ways to measure how a stock moves between two time points:

Simple (arithmetic) return:

$$R_t = (S_t - S_{t-1}) / S_{t-1} = S_t/S_{t-1} - 1$$

Log (continuously compounded) return:

$$r_t = \ln(S_t / S_{t-1})$$

Log returns have crucial mathematical properties that make them the standard in quantitative finance:

Additivity	Multi-period log return = sum of single-period log returns. A 3-day return is simply r1 + r2 + r3. Simple returns don't have this property.
Symmetry	A +10% log return followed by -10% log return returns exactly to the start. Simple returns don't: +10% then -10% gives -1%.
Normality	If prices follow GBM, log returns are normally distributed. This is the foundation of Black-Scholes.
No negative prices	Since S = S0 * exp(r), prices can never go negative regardless of how negative r gets.

Key insight: We always compute log returns from adjusted close prices. Everything downstream, including calibration, simulation, and pricing, builds on this single transformation.

1.3 Annualization

Daily data needs to be scaled to annual terms for comparability. With 252 trading days per year:

```
mu_annual = mean(r_daily) * 252
sigma_annual = std(r_daily) * sqrt(252)
```

The mean scales linearly (252 days of average daily drift) while volatility scales with the square root of time. This square-root scaling comes from the variance of a sum of independent random variables: Var(X1 + X2 + ... + Xn) = n * Var(X) when the Xi are i.i.d., so the standard deviation scales as sqrt(n).

Watch out: The drift mu from log returns is the **log-space** drift. The real-world expected growth rate is mu + sigma^2/2 due to the Ito correction (covered in Chapter 3). This is a common source of bugs.

Chapter 2: Probability and Random Variables

2.1 The Normal Distribution

The normal (Gaussian) distribution is the workhorse of quantitative finance. A random variable Z ~ N(0,1) is called standard normal. Its probability density function (PDF) is the familiar bell curve centered at zero with standard deviation 1.

For a general normal X ~ N(mu, $sigma^2$), we can always write X = mu + sigma * Z where Z is standard normal. This is how we generate normally distributed random numbers in simulation: draw Z from N(0,1), then scale and shift.

2.2 The Log-Normal Distribution

If log returns are normal, then prices are **log-normally distributed**. A random variable Y is log-normal if ln(Y) is normal. Key properties:

Always positive	Y = exp(X) where X is normal, so Y > 0 always. Stock prices can't be negative.
Right-skewed	The distribution has a long right tail. Large upward moves are possible but rare.
Mean	E[Y] = exp(mu + sigma^2/2). Note the sigma^2/2 correction appears here.
Multiplicative	Products of log-normals are log-normal. Multi-period price ratios compose cleanly.

2.3 The Poisson Distribution

The Poisson distribution models the count of rare events in a fixed interval. If events occur at rate lambda per year, the number of events in a small interval dt is:

```
N(dt) ~ Poisson(lambda * dt)
```

In finance, this models sudden jumps: earnings surprises, geopolitical shocks, flash crashes. The Merton jump-diffusion model uses Poisson arrivals for discontinuous price moves that GBM cannot capture.

2.4 Key Statistical Concepts

Expectation E[X]	The probability-weighted average outcome. For returns, this is the drift.
Variance Var(X)	Measures spread. sigma^2 = E[(X - mu)^2]. Volatility sigma is its square root.
Skewness	Asymmetry of the distribution. Negative skew means fat left tail (crash risk).
Kurtosis	Tail thickness. Normal has kurtosis 3. Real returns have excess kurtosis (fat tails).
Correlation	Linear dependence between two variables. Range: [-1, 1]. Used for portfolio modeling.

Why this matters: Real stock returns have fat tails and negative skew. The normal distribution underestimates extreme moves. Jump-diffusion and Heston models exist precisely to capture these features.

Chapter 3: Stochastic Processes and Brownian Motion

3.1 What Is a Stochastic Process?

A stochastic process is a collection of random variables indexed by time. A stock price S(t) is a stochastic process: at each moment t, the price is a random variable whose value depends on all the random shocks that have occurred up to that point.

The goal of quantitative modeling is to specify the **dynamics** of this process: what distribution do small changes dS follow? This is written as a stochastic differential equation (SDE).

3.2 Brownian Motion (Wiener Process)

Brownian motion W(t) is the fundamental building block. It's the continuous-time limit of a random walk. Properties:

Starts at zero	W(0) = 0
Independent increments	W(t) - W(s) is independent of W(u) for u <= s < t
Normal increments	W(t) - W(s) ~ N(0, t - s). The variance grows linearly with time.
Continuous paths	W(t) is continuous but nowhere differentiable (extremely jagged).

In discrete simulation with time step dt, each Brownian increment is:

```
dW = sqrt(dt) * Z, where Z ~ N(0,1)
```

3.3 Stochastic Differential Equations (SDEs)

An SDE describes how a process evolves over infinitesimal time steps. The general form is:

```
dX = a(X, t) dt + b(X, t) dW
```

The first term a(X,t)dt is the **drift** (deterministic trend). The second term b(X,t)dW is the **diffusion** (random shocks). The relative magnitude of drift vs diffusion determines how noisy the process is.

3.4 Ito's Lemma and the Ito Correction

Ito's lemma is the chain rule for stochastic calculus. If S follows an SDE and we apply a function f(S), Ito's lemma tells us what SDE f(S) follows. The key surprise is an extra term that doesn't appear in ordinary calculus.

For stock prices following dS/S = mu*dt + sigma*dW, applying f(S) = ln(S) gives:

$$d(\ln S) = (mu - sigma^2/2)\ dt + sigma\ dW$$

The -sigma^2/2 term is the **Ito correction**. It means the log price drifts slower than the price itself. This correction appears everywhere in quantitative finance:

> **The Ito correction in practice:** When you calibrate GBM from log returns, you estimate mu_log = mu - sigma^2/2. To recover the real-world drift mu, you must add back sigma^2/2. Forgetting this correction causes systematic drift errors proportional to sigma^2.

Chapter 4: Geometric Brownian Motion (GBM)

4.1 The Model

GBM is the simplest and most widely used model for stock prices. It assumes that percentage changes in the stock price are normally distributed with constant drift and volatility.

```
dS = mu * S * dt + sigma * S * dW
```

Where mu is the expected return (drift) and sigma is the volatility. Both are annualized constants. This is the foundation of the Black-Scholes model.

4.2 The Exact Solution

Unlike most SDEs, GBM has a closed-form solution. Applying Ito's lemma to ln(S):

$$S(t) = S(0) * \exp[(mu - sigma^2/2) * t + sigma * W(t)]$$

For simulation, discretizing over a time step dt:

$$S(t+dt) = S(t) * \exp[(mu - sigma^2/2) * dt + sigma * \mathrm{sqrt}(dt) * Z]$$

This is an **exact** discretization, meaning there is no discretization error regardless of the step size dt. This is a unique advantage of GBM. You could simulate a 1-year path in a single step and get the exact terminal distribution.

> **Implementation note:** Because the solution is exact, GBM simulation is purely vectorized: generate a matrix of Z values, compute log increments, cumsum, and exponentiate. No loops over time steps needed.

4.3 Calibrating GBM

Given a series of adjusted close prices, calibration estimates mu and sigma:

Step 1	Compute daily log returns: r_t = ln(S_t / S_{t-1})
Step 2	sigma = std(r_daily, ddof=1) * sqrt(252). Use ddof=1 (Bessel's correction) for an unbiased estimator.
Step 3	mu_log = mean(r_daily) * 252. This is the log-space drift.

Step 4	mu = mu_log + sigma^2/2. Apply Ito correction to get the real-world drift.

This is called the **method of moments**: we match the sample statistics (mean, variance) to the model's theoretical moments.

4.4 Limitations of GBM

GBM is elegant but makes assumptions that real markets violate:

Constant volatility	Real volatility clusters (high-vol periods follow high-vol periods) and changes over time. GBM misses this entirely.
No jumps	Prices sometimes gap discontinuously (earnings, crashes). GBM paths are continuous.
Normal returns	Real returns have fat tails and negative skew. GBM underestimates extreme events.
Volatility smile	GBM implies flat implied volatility across strikes. Markets show a smile/skew.

These limitations motivate the jump-diffusion and Heston models covered in the next two chapters.

Chapter 5: Merton Jump-Diffusion Model

5.1 Motivation: Why Jumps?

Stock prices don't always move smoothly. Earnings surprises, FDA decisions, geopolitical events, and flash crashes cause sudden discontinuous moves. GBM, with its continuous paths, cannot model these jumps. The result is that GBM systematically underprices out-of-the-money options because it underestimates the probability of extreme moves.

5.2 The Model

Merton (1976) extended GBM by adding a Poisson jump component:

```
dS/S = (mu - lambda*k) dt + sigma dW + J dN
```

Where:

lambda	Jump intensity: average number of jumps per year. lambda=2 means ~2 jumps/year.
N(t)	Poisson process with intensity lambda. N(dt) ~ Poisson(lambda*dt) gives the number of jumps in interval dt.
J	Jump size (random). J = exp(Y) - 1 where Y ~ N(mu_j, sigma_j^2). The jump multiplies the price by exp(Y).
k	Jump compensator: k = exp(mu_j + sigma_j^2/2) - 1. This ensures the drift term accounts for the expected jump contribution.
-lambda*k	Drift correction: without this, jumps would add a systematic drift bias. The compensator makes the diffusion+jump drift equal to mu.

5.3 Understanding the Compensator

The compensator is often confusing. Here's the intuition: if jumps on average increase the price (positive mu_j), then the continuous part must drift down by exactly that amount to keep the total expected return equal to mu. Otherwise, a stock with more jumps would have a higher expected return than one without, purely due to the jump component.

$$k = E[\exp(Y)] - 1 = \exp(mu\_j + sigma\_j^2/2) - 1$$

Common bug: Forgetting the compensator -lambda*k in the drift causes a drift error of lambda*k per year. For lambda=2, mu_j=0.01, sigma_j=0.05, this is about 3%/year, enough to bias all downstream pricing.

5.4 Simulation

Each time step combines three components: diffusion, jump count, and jump sizes.

1. Diffusion	Same as GBM: (mu - lambda*k - sigma^2/2)*dt + sigma*sqrt(dt)*Z
2. Jump count	n_jumps = Poisson(lambda * dt). Usually 0 for small dt.
3. Jump sizes	If n_jumps > 0, draw Y_1, ..., Y_n ~ N(mu_j, sigma_j^2). Sum them.
4. Update	S(t+dt) = S(t) * exp(diffusion + sum(Y_i))

The Poisson draws can be vectorized across all simulations using numpy's random.poisson, making this nearly as fast as GBM simulation.

5.5 Calibration

Calibrating jump-diffusion is harder than GBM because we need to separate the continuous and jump components. The approach used in this system:

Step 1	Calibrate GBM first to get baseline mu and sigma.
Step 2	Identify jump candidates: returns exceeding a threshold (e.g., 3*sigma).
Step 3	Estimate lambda from the frequency of threshold-exceeding returns.
Step 4	Estimate mu_j and sigma_j from the magnitudes of identified jumps.
Step 5	Refine via maximum likelihood estimation (MLE) on the full return series, modeling it as a mixture.

A valid calibration result can have lambda close to zero, meaning the data shows no significant jump behavior and the model reduces to GBM. This is fine and expected for low-volatility periods.

Chapter 6: Heston Stochastic Volatility Model

6.1 Motivation: Volatility Is Not Constant

Both GBM and jump-diffusion assume constant volatility sigma. But empirically, volatility exhibits two key behaviors: **clustering** (high-vol days tend to follow high-vol days) and **mean reversion** (extreme volatility eventually returns to a long-run average). The Heston model captures both by making volatility itself a stochastic process.

6.2 The Model

The Heston model uses two coupled SDEs:

$$dS = mu * S * dt + sqrt(v) * S * dW_S$$
$$dv = kappa * (theta - v) * dt + xi * sqrt(v) * dW_v$$

The first equation is like GBM but with stochastic variance v(t) replacing $sigma^2$. The second equation governs how variance evolves over time.

v(t)	Instantaneous variance at time t. Volatility is sqrt(v(t)).
kappa	Mean reversion speed. Higher kappa means variance snaps back faster to theta. Typical: 1-5.
theta	Long-run variance level. The variance v(t) is pulled toward theta over time. Typical: 0.01-0.1.
xi	Vol-of-vol. How much the variance itself fluctuates. Higher xi means more volatile volatility. Typical: 0.1-1.0.
rho	Correlation between price and variance Brownians. Usually negative (-0.3 to -0.9): when prices drop, volatility rises (the leverage effect).
v0	Initial variance. Starting point for the variance process.

6.3 The Feller Condition

$$2 * kappa * theta > xi^2$$

This condition ensures the variance process v(t) never touches zero. When satisfied, the mean-reverting pull toward theta is strong enough relative to the random fluctuations (xi) to keep variance strictly positive. If violated, variance can hit zero, which creates simulation difficulties (square root of zero or negative numbers).

In practice: Calibrated parameters sometimes violate Feller. This doesn't make the model useless, but simulation schemes must handle it. The QE scheme used in this project handles near-zero variance gracefully. The calibrator flags this with a feller_satisfied=False marker.

6.4 Correlated Brownian Motions

The price and variance processes are driven by correlated Brownian motions with correlation rho. To simulate this, we generate two independent standard normals Z1 and Z2, then construct:

$$dW_S = Z_1 * \mathrm{sqrt}(dt)$$
$$dW_v = (\mathrm{rho} * Z_1 + \mathrm{sqrt}(1 - \mathrm{rho}^2) * Z_2) * \mathrm{sqrt}(dt)$$

This is the Cholesky decomposition for 2 variables. The same idea extends to N assets for portfolio simulation (Chapter 13).

6.5 The QE (Quadratic Exponential) Simulation Scheme

Naive Euler discretization of Heston can produce negative variance, which is mathematically invalid. The Andersen QE scheme solves this by using different approximations depending on the current variance level:

Compute psi	psi = xi^2 * (1-exp(-kappa*dt)) / (4*kappa*theta*(exp(-kappa*dt)-1) + ...). This ratio determines which regime we're in.
psi <= psi_crit	Variance is far from zero. Use a quadratic approximation: v(t+dt) = a*(b + Z)^2 where a,b are functions of the current state.
psi > psi_crit	Variance is near zero. Use an exponential approximation that guarantees non-negativity.
psi_crit	The switching threshold, typically 1.5. This is a tuning parameter of the scheme.

Chapter 7: Options — Contracts, Payoffs, and Strategies

7.1 What Is an Option?

An option is a contract that gives the holder the **right, but not the obligation**, to buy or sell an asset at a predetermined price (the **strike**) on or before a specific date (the **expiry**). The seller (writer) of the option takes on the obligation and receives a premium upfront.

Call option	Right to BUY at strike K. Valuable when S > K (stock above strike).
Put option	Right to SELL at strike K. Valuable when S < K (stock below strike).
Strike (K)	The pre-agreed price at which the option can be exercised.
Expiry (T)	The date the option expires. Time to expiry is T years from now.
Premium	The price paid for the option contract. This is what pricing models compute.

7.2 European vs. American Options

A **European** option can only be exercised at expiry. An **American** option can be exercised at any time up to expiry. Our system prices European-style options (and exotic variants), which is the standard for Monte Carlo methods. American options require additional techniques like Longstaff-Schwartz regression.

7.3 Payoff Functions

The payoff is what the option is worth at expiry. This is what we compute from simulated paths:

Option Type	Payoff	Notes
European Call	`max(S_T - K, 0)`	Depends only on terminal price
European Put	`max(K - S_T, 0)`	Depends only on terminal price
Asian Call	`max(avg(S) - K, 0)`	Depends on entire path average
Asian Put	`max(K - avg(S), 0)`	Depends on entire path average
Barrier KO Call	`max(S_T - K, 0) if max(S) < B`	Knocked out if barrier B is breached
Lookback Call	`S_T - min(S)`	Depends on path minimum

Lookback Put	`max(S) - S_T`	Depends on path maximum
Digital Call	`1 if S_T > K, else 0`	Binary payoff

7.4 Moneyness

In-the-money (ITM)	Call: S > K. Put: S < K. The option has intrinsic value right now.
At-the-money (ATM)	S is approximately equal to K. The option could go either way.
Out-of-the-money (OTM)	Call: S < K. Put: S > K. No intrinsic value, only time value.

An ATM call has roughly 50% chance of ending in the money, which is why its delta is approximately 0.5.

7.5 Put-Call Parity

For European options, there is a fundamental relationship between call and put prices:

$$C - P = S_0 - K * \exp(-r * T)$$

This is an arbitrage relationship: if it doesn't hold, you can construct a risk-free profit. It's a powerful check on your pricing implementation: if your MC call and put prices don't satisfy this (within standard error), something is wrong.

Chapter 8: Black-Scholes — The Closed-Form Benchmark

8.1 The Black-Scholes Formula

Black and Scholes (1973) derived a closed-form formula for European option prices under GBM. It assumes the stock follows GBM, there are no transaction costs, continuous trading is possible, and the risk-free rate is constant.

$$C = S_0 * N(d_1) - K * \exp(-rT) * N(d_2)$$

$$d_1 = [\ln(S_0/K) + (r + \text{sigma}^2/2) * T] / (\text{sigma} * \text{sqrt}(T))$$
$$d_2 = d_1 - \text{sigma} * \text{sqrt}(T)$$

Where N(x) is the cumulative standard normal distribution function, r is the risk-free rate, and T is time to expiry in years.

8.2 Risk-Neutral Pricing

A crucial concept: the Black-Scholes formula uses r (risk-free rate) as the drift, not the real-world drift mu. This is **risk-neutral pricing**: in a world where investors are indifferent to risk, all assets earn the risk-free rate on average.

This works because the option price is determined by hedging, not by predicting future returns. The hedging argument eliminates the real-world drift entirely. For Monte Carlo pricing, this means we simulate with drift mu = r (or r - q with dividends), not the calibrated real-world mu.

Critical distinction: Calibrate with real-world drift mu (from historical data). Price with risk-neutral drift r (the risk-free rate). Risk metrics (VaR, etc.) use the real-world mu. Confusing these is one of the most common errors in quant finance.

8.3 Why Monte Carlo If We Have Black-Scholes?

Black-Scholes only works for European options under GBM. Monte Carlo works for:

Exotic options	Asian, barrier, lookback, and other path-dependent options have no closed-form.
Complex models	Jump-diffusion and Heston don't have simple closed-form European prices (Heston has a semi-analytical form, but it's complex).

Multiple assets	Basket options, rainbow options, and portfolio-level pricing require simulation.
Flexibility	New payoffs can be added by writing a single function. No new math derivation needed.

Black-Scholes serves as a benchmark: for European options under GBM, MC prices should converge to the Black-Scholes price as the number of simulations increases.

8.4 Implied Volatility

Given an observed market price for an option, the **implied volatility** is the sigma you'd need to plug into Black-Scholes to reproduce that market price. It's found by numerically inverting the BS formula.

If Black-Scholes were perfectly correct, implied vol would be the same for all strikes and expiries. In reality, implied vol varies, forming the **volatility smile** (higher vol for deep ITM and OTM options) or **volatility skew** (higher vol for OTM puts, reflecting crash fear). This surface is what the Heston model can capture.

Chapter 9: Monte Carlo Simulation for Pricing

9.1 The Core Idea

Monte Carlo pricing is conceptually simple: simulate many possible future paths for the stock price, compute the option payoff on each path, take the average, and discount back to today.

```
V = exp(-r*T) * (1/N) * Sum[ payoff(path_i) ] for i = 1 to N
```

By the law of large numbers, as N grows, this average converges to the true expected payoff under the risk-neutral measure, which is the option's fair price.

9.2 The Algorithm

1. Setup	Choose model (GBM/JD/Heston), set params with risk-neutral drift (mu = r - q).
2. Generate paths	Simulate N price paths from S0 to time T using n_steps time steps.
3. Compute payoffs	Apply the payoff function to each path. For European: use only S_T. For Asian: use path average.
4. Discount	Multiply the average payoff by exp(-r*T) to get present value.
5. Statistics	Compute standard error = std(payoffs) / sqrt(N). Build 95% CI: price +/- 1.96 * SE.

9.3 Convergence and Standard Error

The standard error of a Monte Carlo estimate decreases as 1/sqrt(N). This means:

N = 1,000	SE is proportional to 1/sqrt(1000) = 0.032. Rough estimate.
N = 10,000	SE is proportional to 1/sqrt(10000) = 0.01. Decent precision.
N = 100,000	SE is proportional to 1/sqrt(100000) = 0.003. Good precision.
10x more paths	Only 3.16x improvement in accuracy. Diminishing returns.

This slow convergence is why variance reduction techniques (Chapter 10) are so important: they can achieve the same accuracy with far fewer paths.

9.4 Common Random Numbers (CRN)

When computing finite-difference Greeks (Chapter 11), we bump a parameter and reprice. If each repricing uses different random numbers, the noise overwhelms the signal. CRN means using the exact same random draws (same seed) across all bumped prices. The random noise cancels in the difference, leaving a clean sensitivity estimate.

Implementation: Pass a z_extern array to the simulator: pre-generated random numbers that are reused across all Greeks calculations. This is why our simulator has a z_extern interface.

Chapter 10: Variance Reduction Techniques

10.1 Why Variance Reduction?

Naive MC converges at O(1/sqrt(N)). To get 10x better accuracy, you need 100x more paths. Variance reduction techniques reduce the variance of the estimator without adding paths, effectively getting more accuracy for free.

10.2 Antithetic Variates

For every random draw Z, also simulate with -Z. If Z produces a high path, -Z produces a low path. Averaging the two reduces variance because the payoffs are negatively correlated.

```
V = exp(-rT) * (1/N) * Sum[ (payoff(Z_i) +
        payoff(-Z_i)) / 2 ]
```

How it works	Each (Z, -Z) pair produces a pair average. The pair averages have lower variance than individual payoffs.
Variance reduction	Typically 2x-25x reduction depending on the payoff structure.
Cost	Essentially free: -Z costs nothing to compute, and you get the same number of independent estimates from N/2 pairs.
SE formula	Important: use the variance of pair averages, not individual payoffs. The i.i.d. formula overstates SE by ~1.8x for antithetic paths.

10.3 Stratified Sampling

Instead of drawing Z values purely at random, divide the N(0,1) quantile space into equal strata and draw one sample from each stratum. This guarantees better coverage of the distribution.

How it works	Divide [0,1] into N equal intervals. Draw one uniform U_i from each interval. Transform: Z_i = Phi_inv(U_i).
Variance reduction	Can achieve 10x-100x reduction. Particularly effective for tail-sensitive payoffs.
Intuition	Removes the sampling noise in how well the draws represent the true distribution. Every region gets exactly its fair share.

In our system: Both techniques are implemented in generate_samples(). Antithetic gives ~22x reduction, stratified ~81x reduction on European option pricing. The combination makes 10k paths competitive with 100k+ naive paths.

Chapter 11: The Greeks — Sensitivity Analysis

11.1 What Are the Greeks?

The Greeks measure how an option's price changes when one input changes while holding everything else constant. They are the partial derivatives of the option price with respect to various inputs. Traders use Greeks to understand and hedge their risk exposure.

11.2 The Five Main Greeks

Delta

Sensitivity to stock price change. Delta = dV/dS. A delta of 0.6 means the option price moves $0.60 for every $1 move in the stock. ATM calls have delta near 0.5. Used for hedging: hold -delta shares to neutralize price risk.

Gamma

Rate of change of delta. Gamma = d^2V/dS^2. High gamma means delta changes quickly, requiring frequent rebalancing. Highest for ATM options near expiry.

Vega

Sensitivity to volatility. Vega = dV/d(sigma). Positive for all vanilla options: higher vol means higher option value (more chance of a big move in your favor).

Theta

Time decay. Theta = dV/dT (negative). Options lose value as time passes because there's less time for favorable moves. ATM options have the highest theta.

Rho

Sensitivity to interest rates. Rho = dV/dr. Usually the smallest Greek. Positive for calls (higher r makes the strike cheaper in PV terms), negative for puts.

11.3 Bump-and-Revalue Method

For Monte Carlo pricing, we compute Greeks numerically via finite differences:

```
Delta = [V(S+h) - V(S-h)] / (2h) (central difference)
```

We bump the input parameter by a small amount h, reprice using the same random numbers (CRN), and compute the slope. Central differences are more accurate than forward differences.

Theta trap: When bumping T for theta, you must hold n_steps fixed and adjust dt = T/n_steps. If you hold dt fixed and reduce n_steps, the path structure changes and CRN breaks, producing pure noise instead of a sensitivity estimate.

Chapter 12: Risk Metrics — VaR, CVaR, and Beyond

12.1 Value at Risk (VaR)

VaR answers: what is the worst loss I can expect with a given confidence level over a given time horizon?

```
VaR(alpha) = -Percentile(P&L distribution, alpha)
```

For example, a 95% VaR of $10,000 means: there is a 5% chance of losing more than $10,000 over the holding period. In our system, VaR is computed directly from simulated P&L;: sort the terminal P&L; values and take the alpha-th percentile.

> **Convention note:** We report VaR as raw P&L; percentiles. A negative number means a loss. Some textbooks flip the sign (report VaR as a positive number representing loss magnitude). Be aware of the sign convention when comparing.

12.2 Conditional VaR (CVaR / Expected Shortfall)

VaR tells you the threshold loss, but nothing about how bad things get beyond that threshold. CVaR (also called Expected Shortfall) answers: *given that we're in the worst alpha% of outcomes, what's the average loss?*

```
CVaR(alpha) = E[P&L | P&L <= VaR(alpha)]
```

CVaR is always worse than VaR (more negative). It captures tail risk that VaR ignores. Regulators increasingly prefer CVaR because it is a **coherent risk measure** (it has nice mathematical properties like subadditivity: portfolio CVaR is at most the sum of individual CVaRs).

12.3 Maximum Drawdown

The maximum drawdown measures the worst peak-to-trough decline along a price path:

```
MaxDD = max over t [ (peak_t - S_t) / peak_t ]
```

Where peak_t is the running maximum up to time t. A drawdown of 30% means at some point, the portfolio lost 30% from its highest value. Unlike VaR which looks only at the terminal value, drawdown captures the worst point along the entire path.

12.4 Sharpe Ratio

```
Sharpe = (E[r] - r_f) / sigma[r]
```

The Sharpe ratio measures risk-adjusted return: how much excess return (above the risk-free rate) do you earn per unit of volatility? A Sharpe above 1 is generally considered good. From simulations, we compute the mean and standard deviation of simulated returns to estimate this.

Chapter 13: Portfolio Simulation and Correlation

13.1 Why Correlation Matters

A portfolio of multiple stocks can't be simulated by running each stock independently. Their returns are correlated: tech stocks tend to move together, oil stocks move with energy prices. Ignoring correlation underestimates portfolio risk during market-wide sell-offs.

13.2 The Correlation Matrix

Given N stocks, the correlation matrix is N x N, where entry (i,j) is the Pearson correlation between the log returns of stock i and stock j. The diagonal is always 1 (each stock is perfectly correlated with itself). The matrix is symmetric.

13.3 Cholesky Decomposition

To generate correlated random variables from independent ones, we use the Cholesky decomposition. Given a correlation matrix C, find the lower triangular matrix L such that $C = L * L^T$. Then if Z is a vector of independent standard normals:

```
X = L * Z gives correlated normals with correlation matrix C
```

This is the same technique used in Chapter 6 for correlated Brownian motions in Heston, generalized to N dimensions.

> **Singular correlation matrices:** If two assets are perfectly correlated (rho = 1) or the matrix is otherwise singular, Cholesky decomposition fails. Our system adds a tiny ridge regularization (1e-10 * I) to handle this edge case.

13.4 Portfolio Value Simulation

Given N assets with weights w1, ..., wN and initial portfolio value V0, the simulation proceeds:

1. Compute	Historical correlation matrix from overlapping log return series.
2. Cholesky	Decompose correlation matrix: C = L * L^T.
3. Generate	For each time step, draw N independent normals, multiply by L to get correlated normals.

4. Simulate	Each asset evolves under its own model and parameters, using its correlated Brownian increment.
5. Aggregate	Portfolio value = Sum(w_i * S_i(t) / S_i(0)) * V0 at each time step.

Chapter 14: Model Calibration from Market Data

14.1 What Is Calibration?

Calibration is the process of estimating model parameters from observed data so that the model reproduces the statistical properties of the data. It bridges the gap between abstract mathematical models and real market behavior.

There are two philosophies of calibration:

Historical calibration	Fit parameters to historical returns. This captures real-world dynamics (the P-measure). Used for risk metrics, scenario analysis, and understanding the data.
Market-implied calibration	Fit parameters to current option prices. This captures the market's risk-neutral view (the Q-measure). Used for pricing and hedging.

Our system uses historical calibration for all three models. Market-implied calibration (fitting Heston to the implied vol surface) is a planned future enhancement.

14.2 Method of Moments (GBM)

The simplest calibration approach: match the sample moments to the theoretical moments. For GBM, the first two moments of log returns give us mu and sigma directly. This is fast, closed-form, and robust.

14.3 Maximum Likelihood Estimation (Jump-Diffusion)

MLE finds the parameters that maximize the probability of observing the data. For jump-diffusion, the log-likelihood of each return r_t is the log of a mixture density: a normal component (no jump) and Poisson-weighted normal components (k jumps). We maximize the total log-likelihood using numerical optimization.

14.4 Optimization-Based Calibration (Heston)

Heston calibration minimizes the distance between model-implied and observed volatility term structure at multiple horizons. The loss function is:

```
L(kappa, theta, xi, rho, v0) = Sum[ (sigma_model(T_i) - sigma_observed(T_i))^2 ]
```

This is solved using scipy.optimize.minimize with the SLSQP method, which supports parameter constraints (kappa > 0, theta > 0, etc.) and the Feller condition as an inequality constraint.

> **Validation strategy:** The strongest calibration test is a round-trip: generate synthetic data from known parameters, calibrate, and check that recovered parameters match the originals. This catches bugs that unit tests on real data cannot.

14.5 Goodness of Fit

How do we know if a model fits well? Several metrics help:

Log-likelihood	Higher is better. The probability of the observed data under the fitted model.
AIC	Akaike Information Criterion: -2*LL + 2*k (k = number of parameters). Penalizes complexity. Lower is better.
BIC	Bayesian Information Criterion: -2*LL + k*ln(n). Stronger complexity penalty than AIC. Lower is better.
Jarque-Bera test	Tests whether residuals are normal. Low p-value means the model misses some structure in the data.

Chapter 15: Stress Testing and Scenario Analysis

15.1 What Is Stress Testing?

Stress testing asks: *how would our portfolio perform under extreme but plausible conditions?* Instead of relying only on the calibrated parameters (which reflect normal market conditions), we override specific parameters to simulate crisis scenarios.

15.2 Scenario Families

Our system provides two main scenario families, each with model-specific parameter overrides:

Vol Spike	Sudden increase in volatility (like VIX doubling). For GBM: multiply sigma by 2-3x. For Heston: increase v0 and theta. Simulates uncertainty shocks.
Market Crash	A severe downturn. For GBM: shift drift negative and increase sigma. For JD: add a large negative jump. For Heston: increase v0, make rho more negative. Simulates 2008 or COVID-like events.

The scenarios are keyed by (scenario_name, model_type), so each model gets parameter overrides that make physical sense for its dynamics. A vol spike in Heston means different things than in GBM.

15.3 Interpreting Stress Results

The value of stress testing is not in predicting the future but in understanding vulnerabilities. Comparing baseline VaR to stressed VaR reveals how sensitive the portfolio is to different types of shocks. A portfolio that looks safe under normal conditions might have catastrophic tail risk under a vol spike scenario.

Appendix A: Mathematical Reference

A.1 Key Formulas

Log return	`r_t = ln(S_t / S_{t-1})`
Annualized vol	`sigma_annual = sigma_daily * sqrt(252)`
Ito correction	`mu = mu_log + sigma^2 / 2`
GBM exact solution	`S(t) = S(0) * exp[(mu - sigma^2/2)*t + sigma*W(t)]`
JD compensator	`k = exp(mu_j + sigma_j^2/2) - 1`
Feller condition	`2*kappa*theta > xi^2`
BS d1	`d1 = [ln(S/K) + (r + sigma^2/2)*T] / (sigma*sqrt(T))`
BS d2	`d2 = d1 - sigma*sqrt(T)`
Put-call parity	`C - P = S - K*exp(-rT)`
MC price	`V = exp(-rT) * mean(payoffs)`
MC standard error	`SE = std(payoffs) / sqrt(N)`
Central difference	`dV/dX = [V(X+h) - V(X-h)] / (2h)`
Cholesky	`C = L * L^T, then X = L * Z for correlated normals`
Sharpe ratio	`SR = (E[r] - r_f) / sigma[r]`

A.2 Notation Summary

`S, S(t), S_T`	Stock price (current, at time t, at expiry)
`K`	Strike price
`T`	Time to expiry (years)
`r`	Risk-free interest rate (annualized)
`q`	Continuous dividend yield
`mu`	Expected return / drift (annualized, real-world)
`sigma`	Volatility (annualized)
`W(t), dW`	Brownian motion, Brownian increment
`Z`	Standard normal random variable

`N(x)`	Cumulative standard normal CDF
`dt`	Time step size (e.g., 1/252 for daily)
`lambda`	Jump intensity (jumps per year)
`mu_j, sigma_j`	Jump mean and jump volatility (log scale)
`kappa`	Heston mean reversion speed
`theta`	Heston long-run variance
`xi`	Heston vol-of-vol
`rho`	Correlation (price and variance Brownians, or between assets)
`v(t), v0`	Heston instantaneous variance (at t, initial)
`VaR`	Value at Risk
`CVaR / ES`	Conditional VaR / Expected Shortfall

This course was designed as a companion to the Monte Carlo Options Pricing System. Each concept maps directly to a component in the codebase.